

Build and Verify a Path-Geometry Package

Functions, tests, NumPy arrays, and reproducible evidence

Complete four numerical functions in the package `ap_week02_path`. They live in the module `ap_week02_path.path_geometry`, measure and interpolate two-dimensional paths, and are imported by protected callers. Your implementation must preserve coordinate order, reject invalid public input, leave caller-owned arrays unchanged, and produce evidence that another person can reproduce from one identified Git revision.

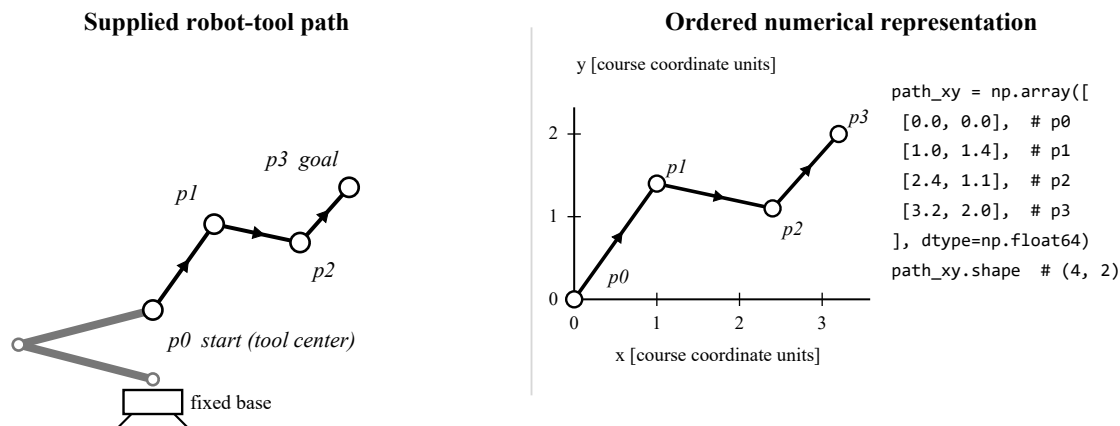


Figure A2.1. The same supplied robot-tool path has a physical interpretation and an ordered numerical representation.

The robot-tool illustration gives the supplied coordinates an engineering context; it is not a robot execution task. You do not generate a route, check collisions, assign timing, implement a controller, or write visualization code. The provided display program imports your numerical functions and converts their returned arrays into an animated SVG. Numerical tests, not the animation alone, verify the public contract.

Provided and fixed - do not edit	Student-editable regions
package structure, <code>PathInputError</code> , and public API	TODO 1-4 and internal helpers in <code>path_geometry.py</code>
published tests, protected scripts, demo, and display	at least six tests in <code>test_student_evidence.py</code>
repository and submission contract	responses in <code>artifacts/engineering_note.md</code>

Manual edits and generated outputs. Manually edit only `src/ap_week02_path/path_geometry.py`, `tests/test_student_evidence.py`, and `artifacts/engineering_note.md`. In `path_geometry.py`, complete TODO 1-4 without changing `PathInputError`, public names, signatures, units, or shapes. Generated JSON, SVG, and wheel files are required outputs, not editable regions: create them with the protected commands, commit them, and do not hand-edit them.

Step 1 - Read the contract, create the private repository, and verify the environment

Read the Week 2 Reading, this Assignment, the repository README.md, and PROTECTED_FILES.md before editing. Then open the course template, select Use this template, and create applied-programming-w02-`<student-id>` under your account with Private visibility. Invite SSUMechE as collaborator, clone your private repository, and confirm that `origin` points to it.

```
https://github.com/SSUMechE/applied-programming-2026-week02-starter
```

```
conda activate applied-programming-2026
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
git remote -v
```

Do not edit source code until the environment verifier ends with `[PASS] Week 2 path-geometry environment is ready.` and both remote lines show your private repository. The package directory is `src/ap_week02_path/`; the editable numerical module has `import path ap_week02_path.path_geometry`. The protected demo and scripts are separate callers of that module.

Step 2 - Review the data flow and record the intentional baseline

```
python -m ap_week02_path.review_data_flow
python scripts/run_baseline.py
```

The review program traces tuple, list, slicing, unpacking, zip, loop, comprehension, and array conversion without revealing the four solutions. The starter baseline must be exactly 42 failed, 6 passed. Before editing the TODO file, copy the summary and first failing pytest test node ID into section 0 of `artifacts/engineering_note.md`.

Step 3 - Complete TODO 1 and TODO 2: the segment functions

Order	Required behavior	Focused checkpoint
TODO 1	<code>segment_length</code> : validate two (2,) points; return one nonnegative Python float	3-4-5 result 5.0; zero length accepted
TODO 2	<code>interpolate_segment</code> : integer $M \geq 2$; return float64 shape (M,2)	for M=5, shape (5,2) and exact endpoints

```
python -m pytest -q tests/test_published_contract.py -k segment_length -x
python -m pytest -q tests/test_published_contract.py -k interpolation -x
```

Step 4 - Complete TODO 3 and TODO 4: scalar and batch path lengths

Continue only when checkpoint A passes. Implement the scalar path function before the batch function so that the scalar result can serve as an independently structured comparison.

Order	Required behavior	Focused checkpoint
TODO 3	<code>path_length</code> : finite shape $(N, 2)$, $N \geq 2$; sum consecutive Euclidean lengths	direct distance <code>5.0</code> ; corner path length <code>7.0</code>
TODO 4	<code>path_lengths</code> : finite shape $(B, N, 2)$, $B \geq 1$, $N \geq 2$; NumPy operations only; return $(B,)$	reject empty batch/path axes; verify scalar-batch agreement, row order, and no mutation

```
python -m pytest -q tests/test_published_contract.py -k "path_length and not path_lengths" -x
python -m pytest -q tests/test_published_contract.py -k path_lengths -x
python -m pytest -q tests/test_published_contract.py
```

The published suite must report 48 passed when the four functions are complete. The batch function must not use a Python `for`, `while`, or comprehension to iterate over paths. It must reject an empty batch axis ($B=0$) or an empty/one-waypoint path axis ($N<2$) with `PathInputError`. Every public function rejects wrong representation, wrong shape, unsupported type, object-dtype arrays, or non-finite values and leaves caller-owned data unchanged. These checks constitute numerical contract verification; a plausible picture does not replace them.

Fixed public rule	Required meaning
coordinate order	each point is ordered (x, y)
<code>num_samples</code>	Python or NumPy integer at least 2; Boolean rejected
batch/path sizes	for $(B, N, 2)$, require $B \geq 1$ and $N \geq 2$; reject empty axes
side effects	no print, plot, file writing, global mutation, or visualization call
invalid input	raise <code>PathInputError</code> ; do not reshape, clip, or repair silently

Complete these core implementation checkpoints before beginning the independent evidence work on the next page. This order keeps the scalar result available as a reference for the batch implementation.

Step 5 - Add independent numerical evidence and build the package

The assignment is not complete when the published suite first passes. Add at least six top-level `test_...` functions to `tests/test_student_evidence.py` using inputs different from the published examples. Collectively include normal, boundary, invalid, endpoint, translation, and batch-or-ownership evidence.

Use these category prefixes: `test_normal_`, `test_boundary_`, `test_invalid_`, `test_endpoint_`, `test_translation_`, and one of `test_batch_`, `test_ownership_`, `test_no_mutation_`, or `test_scalar_batch_`. In `artifacts/engineering_note.md`, keep every question and `<!-- STUDENT RESPONSE -->` marker; replace only the placeholder comment below each marker.

Evidence	Minimum independent check
normal	a new segment, interpolation, or path result
boundary	zero-length segment or minimum valid sample/path size
invalid	wrong shape, non-finite value, invalid sample count, or unsupported type
endpoint	first and last interpolated rows equal start and goal
translation	a common coordinate offset preserves path length
batch or ownership	scalar-batch equivalence, row permutation, or no mutation

For the required normal, translation, and batch-or-ownership evidence, use `np.isclose` or `np.allclose` with explicit `rtol` and `atol`. Exact equality may be used for integer shapes, exact Boolean conditions, and a boundary invariant that the public contract guarantees exactly, such as zero length for identical endpoints.

```
python -m pytest -q
python -m ap_week02_path.demo
python scripts/generate_artifacts.py
python -m build --wheel --no-isolation
python scripts/check_submission.py
```

The demo must show the scalar segment, interpolation shape and endpoints, one-path length, batch lengths, scalar-batch agreement, and unchanged inputs. The canonical artifact command imports the numerical module and regenerates `artifacts/path_geometry_summary.json` and `artifacts/path_geometry_preview.svg` from the current code. The SVG is display evidence, not physical execution evidence. Answer all prompts in `artifacts/engineering_note.md`, then build the wheel before running the final checker.

What the final checker establishes. `check_submission.py` collects and runs the independent tests, checks the six evidence categories and completed note, recomputes the JSON/SVG evidence, and clean-installs the single wheel under `dist/`. A PASS applies only to these explicit checks.

Step 6 - Freeze and submit one reproducible Git revision

```
python -m pytest -q
git status --short
git add src/ap_week02_path/path_geometry.py tests/test_student_evidence.py artifacts dist
git commit -m "Complete Week 2 path-geometry package"
git push
git status --short
git rev-parse HEAD
```

After the push, `git status --short` must print no entries. Record the complete 40-character commit identifier. A branch name, screenshot, or the phrase 'latest version' does not identify the source revision that should be evaluated.

Submit the repository identity through LMS

LMS field	Required value
1	private GitHub repository URL
2	full 40-character commit identifier from <code>git rev-parse HEAD</code>
3	confirmation that SSUMechE has active collaborator access

Example of a complete LMS entry

The values below are fictional. Replace every value with the URL, commit identifier, and access status of your own repository.

```
Repository URL: https://github.com/ssu-example-student/applied-programming-w02-20261234
Full commit ID: 7f3c9a1d5e8b4c2f6a0d9e3b1c7f4a8d2e6b0c5f
SSUMechE access: Active collaborator access confirmed (not Pending)
```

Copy the repository URL from the main page of your private Week 2 repository. Obtain the commit identifier only after the final push by running `git rev-parse HEAD`. In GitHub repository settings, confirm that SSUMechE has active access rather than a pending invitation.

Do not submit the public course-template URL, a local folder path, a branch name, `latest`, a seven-character short hash, or an unpushed commit.

Never submit a password, personal access token, two-factor code, or recovery code. The LMS timestamp and stated commit define the grading boundary. Later commits are not included unless a new commit identifier is submitted under the announced policy.

Completion conditions. environment verifier PASS; baseline and first failure recorded in engineering-note section 0; TODO 1-4 complete; 48 published tests pass; at least six independent student tests pass; JSON and SVG artifacts regenerated; engineering note complete; wheel clean-install check passed; private origin correct; SSUMechE access active; pushed revision clean; LMS fields submitted.

Your final result is therefore a tested numerical package plus its reproducible evidence, not only four completed function bodies.